

DA2304: Introduction to Computer Systems

Assignment 1: Memory Architecture, Hardware Multipliers, and Toolchains

Part 1 — Registers and Memory Architecture

Name: Mohammed Khaja Ghouse Mohiuddin

Roll No: DA24B015

REPORT

1 Exercise 1: The 1-Bit Register (Bit) vs. DFF

1.1 Implementation

A Data Flip-Flop (DFF) is a primitive sequential element whose behaviour is:

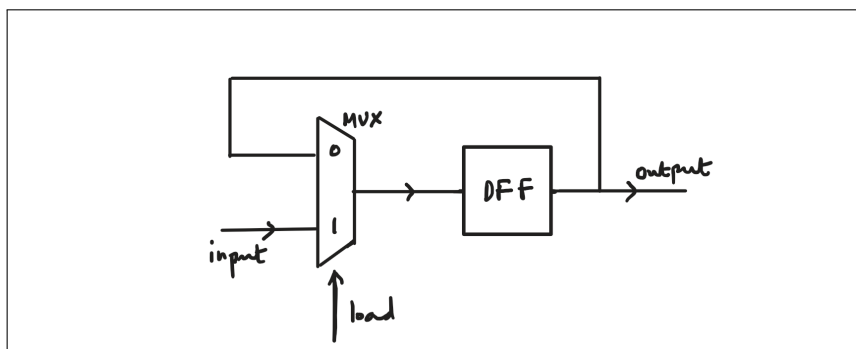
$$\text{out}(t + 1) = \text{in}(t)$$

It simply captures the input on every clock edge and presents it as the output. To give ability to selectively load, we can simply wrap the DFF with a multiplexer, leading to a 1 Bit register `Bit.hdl`

```

1 CHIP Bit {
2     IN in, load;
3     OUT out;
4
5     PARTS:
6     Mux(a=feedback, b=in, sel=load, out=muxout);
7     DFF(in=muxout, out=feedback, out=out);
8 }

```



Circuit Diagram

Justification: when `load = 0`, the Mux will select the feedback path (`a = feedback`), so the DFF will reload its own previous output (ie., the value is held.)

When `load = 1`, the Mux selects the external input (`b = in`), so the DFF will capture the new data on the next clock tick.

1.2 Theoretical Comparison

A DFF does not have the ability to preserve its previous state. Its value gets overridden by the incoming input at every tick. Whereas a 1-Bit Register has the ability to preserve its previous state by controlling the `load` signal. When `load = 0`, the register holds its current value until we fire `load`; only when `load = 1`, it accepts new data. Hence the fundamental difference is selectivity

1.3 1-Bit Counter Implementation

A 1-bit counter sequence goes as $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \dots$ we are always outputting **NOT** of the current state at the next time step. Hence we will always take the output of `Bit` and route it back in with a **NOT** operation. ie., we should set `load = 1` permanently and add the extra **NOT** gate in the system. Note that we have to use a MUX to initialize the state to Zero

```

1 CHIP Counter1Bit {
2     IN initialize;
3     OUT out;
4
5     PARTS:
6     Not(in=current, out=toggled);
7     Mux(a=toggled, b=false, sel=initialize, out=next);
8     Bit(in=next, load=true, out=current, out=out);
9 }

```

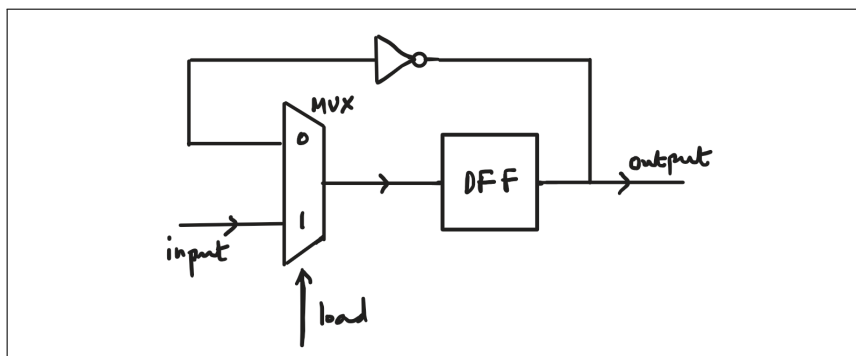


Figure 1: 1-Bit Counter — Circuit Diagram

This counter is simple, since the load doesn't need to change. Hence a bare DFF plus an negation is sufficient.

2 Exercise 2: The 16-Bit Register

2.1 Implementation

we can implement a 16-Bit Register by wiring 16 1-Bit Register (`Bit`) in parallel. They all share the same `load` signal, since we operate on the word as a whole.

```

1  CHIP Register {
2      IN in[16], load;
3      OUT out[16];
4
5      PARTS:
6          Bit(in=in[0], load=load, out=out[0]);
7          Bit(in=in[1], load=load, out=out[1]);
8          Bit(in=in[2], load=load, out=out[2]);
9          Bit(in=in[3], load=load, out=out[3]);
10         Bit(in=in[4], load=load, out=out[4]);
11         Bit(in=in[5], load=load, out=out[5]);
12         Bit(in=in[6], load=load, out=out[6]);
13         Bit(in=in[7], load=load, out=out[7]);
14         Bit(in=in[8], load=load, out=out[8]);
15         Bit(in=in[9], load=load, out=out[9]);
16         Bit(in=in[10], load=load, out=out[10]);
17         Bit(in=in[11], load=load, out=out[11]);
18         Bit(in=in[12], load=load, out=out[12]);
19         Bit(in=in[13], load=load, out=out[13]);
20         Bit(in=in[14], load=load, out=out[14]);
21         Bit(in=in[15], load=load, out=out[15]);
22 }

```

3 Exercise 3: Byte-Level Access

Answers:

1. It depends. If we want finer control over the individual 8-bit halves, we would ideally want two different loads controlling each of the 8-bit registers. In this case, the signals defined in the monolithic 16-bit register won't be sufficient. On the other hand, if we don't want finer control, it becomes virtually the same thing as a monolithic 16-bit register, in which case the split won't serve any new purpose.

Since the purpose of the exercise is to have independent byte-level control, the answer is **No**; the existing signals are not sufficient.

2. Extra signal needed:

Since we are dealing with the case of having individual byte control, we can define a new **byte-select** signal (`sel`) that specifies which of the two bytes to load: for example:

- `sel = 0` → load the low byte (bits 0–7)
- `sel = 1` → load the high byte (bits 8–15)

Note: The input will narrow to 8 bits (`in[8]`), since only one byte is loaded at a time. we can use a `DMux` to steer the `load` signal to the correct register based on `sel`.

3. Advantages and disadvantages of 8-bit vs 16-bit loading:

8-bit loading lets us update one byte without touching the other, and uses a narrower data bus which means less wiring. But it takes two clock cycles to write a full 16-bit word, and the control logic is more complex because of the added `sel` signal.

16-bit loading can write a full word in one clock cycle and the control is simpler since there is no byte selection needed. But we cannot update just one byte on its own, it always overwrites the entire word, and we need a full 16-bit wide data bus which means more wires.

4 Exercise 4: Folding into RAM8

1. RAM8 Architecture:

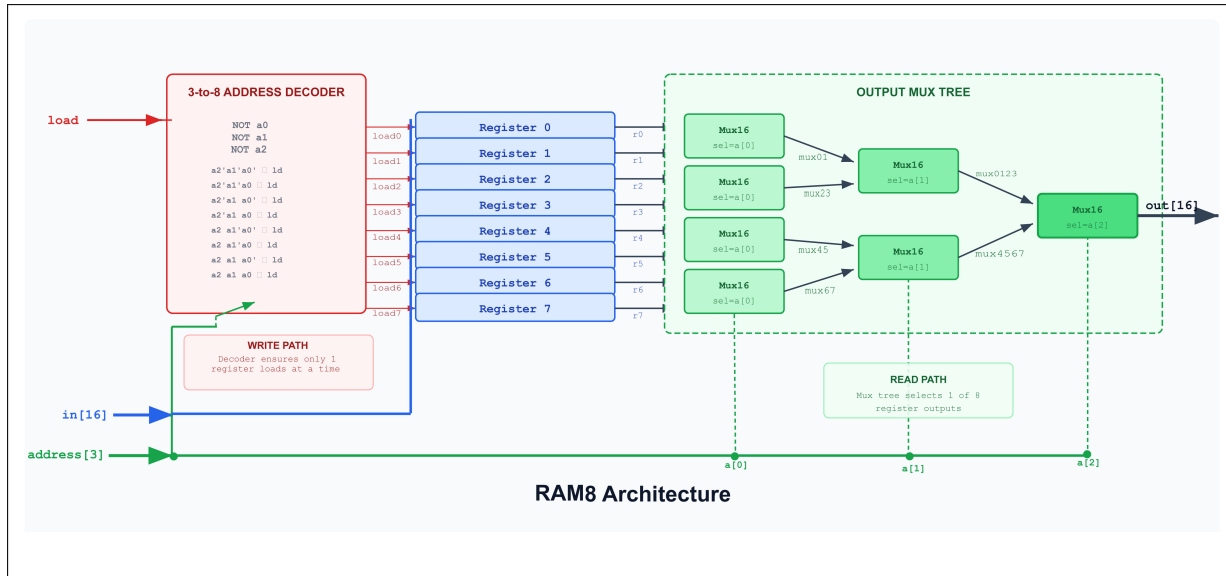
The architecture is as follows: Decode `address[3]` → update registers according to the load → output the corresponding register

So for this, I have implemented a `3AddressDecoder8` Which is built using the simpler `DMux` To route the `load` signal to exactly one of the 8 registers based on the `address` (For the first step), and a `8AddressOut`, which also built using the simpler `Dmux16` to select the output of the addressed register. (the last step)

```

1  CHIP RAM8 {
2      IN in[16], load, address[3];
3      OUT out[16];
4      PARTS:
5          //address routing
6          3AddressDecoder8(in=load, sel=address, a=load0, b=load1, c=
              load2, d=load3, e=load4, f=load5, g=load6, h=load7);
7          //update registers
8          Register(in=in, load=load0, out=r0);
9          Register(in=in, load=load1, out=r1);
10         Register(in=in, load=load2, out=r2);
11         Register(in=in, load=load3, out=r3);
12         Register(in=in, load=load4, out=r4);
13         Register(in=in, load=load5, out=r5);
14         Register(in=in, load=load6, out=r6);
15         Register(in=in, load=load7, out=r7);
16         //output selection
17         8AddressOut(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7,
              sel=address, out=out);
18 }

```



2. IN and OUT signals:

- IN: in[16] (16-bit data input), load (to enable writing), address[3] (to select the register)
- OUT: out[16] (16-bit data output)

3. Address bits required:

At least $\log_2(8) = 3$ bits are required to uniquely index each of the 8 registers

5 Exercise 5: Memory Sizing

1. 1 Kilobit (Kb): $2^{10} = 1,024$ bits.
2. 1 Kilobyte (KB): $1 \text{ KB} = 2^{10} \times 8 = 2^{13} = 8,192$ bits.
3. 16 KB: $16 \text{ KB} = 2^{13} \times 16 = 2^{17} = 131,072$ bits.

6 Exercise 6: Scaling Up Memory

Target: 1 KB of memory = 2^{13} bits.

1. RAM8 chips:

Each RAM8 stores $8 \times 16 = 128 = 2^7$ bits.

$$\frac{2^{13}}{2^7} = 2^6 = 64 \text{ units}$$

2. RAM16 chips:

Each RAM16 stores $16 \times 16 = 256 = 2^8$ bits.

$$\frac{2^{13}}{2^8} = 2^5 = 32 \text{ units}$$

3. RAM64 chips:

Each RAM64 stores $64 \times 16 = 1,024 = 2^{10}$ bits.

$$\frac{2^{13}}{2^{10}} = 2^3 = 8 \text{ units}$$

7 Exercise 7: Address Line Management

for 1KB

1. Using RAM8 (64 units, derived in Exercise 6):

- $\log_2(64) = 6$ address bits to select each RAM8 block (bits 9...3)
- $\log_2(8) = 3$ internal address bits to select a register within the block (bits 2...0)

Using RAM64 (8 units, derived in Exercise 6):

- $\log_2(8) = 3$ address bits to select each RAM64 block (bits 9...6)
- $\log_2(64) = 6$ internal address bits to select a register within the block (bits 5...0)

2. No. The total number of address bits depends on the total number of atomic memory units (16 bit registers in our case):

$$\text{Total bits} = \log_2(\text{total registers}) = \log_2(512) = 9$$

Building Block	Block-select bits	Internal bits	Total
RAM8	6	3	9
RAM64	3	6	9

This remains 9 regardless of the building block choice

3. RAM16K implementation:

RAM16K = 16,384 registers $\Rightarrow$ 14 address bits. We build it hierarchically starting from Bit to Register to RAM8 to RAM64 to RAM512 to RAM4K to RAM8K, using the previously defined 3AddressDecoder8 and 8AddressOut and newly written 2AddressDecoder4 and 4AddressOut

```

1  CHIP RAM16K {
2      IN in[16], load, address[14];
3      OUT out[16];
4      PARTS:
5          2AddressDecoder4(in=load, sel=address[12..13], a=load0, b=
              load1, c=load2, d=load3);
6          RAM4K(in=in, load=load0, address=address[0..11], out=r0);
7          RAM4K(in=in, load=load1, address=address[0..11], out=r1);
8          RAM4K(in=in, load=load2, address=address[0..11], out=r2);
9          RAM4K(in=in, load=load3, address=address[0..11], out=r3);
10         4AddressOut(a=r0, b=r1, c=r2, d=r3, sel=address[12..13], out
              =out);
11 }

```

RAM16K.hdl — 16K×16-bit Memory

8 Extra Credit: Architectural Impact

1. Architectural and performance impact: RAM4K vs RAM8

When we use small blocks like RAM8, we end up needing a very deep MUX/DMUX selection tree to pick the right block. With large blocks like RAM4K, this tree is much shallower. Each level of the tree adds gate delay, so more levels means slower memory access.

This becomes a big problem for AI workloads. During neural network training and inference, the system reads weight matrices sequentially, going through thousands of consecutive addresses one after another. With RAM4K, a single block can handle 4,096 consecutive reads without ever leaving the block. With RAM8, we hit a block boundary every 8 reads, and each time the whole selection tree has to re-evaluate. When you are doing millions of reads for one matrix multiply, all that switching overhead really adds up.

Another advantage of large blocks is that parallel access becomes easy. We can have a few independent RAM4K banks feeding multiple multiply-accumulate units at the same time. This is basically how modern GPUs and AI accelerators work. I don't think it is practical to co-ordinate thousands of RAM8 blocks for parallel access.

So RAM4K gives us a shallower tree, lower latency, fewer block boundary crossings, and easy parallelism, which are favoured by AI workloads

2. Manufacturing cost: $1K \times \text{RAM8}$ vs $8 \times \text{RAM1K}$ for 16KB

The $1K \times \text{RAM8}$ approach is much more expensive. Both designs store the same amount of data (8,192 registers), so the storage itself costs the same. The problem is everything else we need to make the blocks work together

With 1,024 RAM8 blocks, we need to be able to pick exactly one block out of 1,024. That requires a 10-bit decoder built from around 10,240 AND gates. On the output side, we need to funnel the data from 1,024 blocks down to a single 16-bit output, which takes a massive multiplexer tree of around 16,368 gates. So just to route signals around, we are spending roughly 26,000 gates without storing a single extra bit of data

With 8 RAM1K blocks, we only need to pick one out of 8. A 3-bit decoder does this with about 24 gates, and the output multiplexer needs about 112 gates. That is around 136 gates total, nearly 200 times less than the RAM8 approach

On top of that, soldering 1,024 chips onto a circuit board versus just 8 means more PCB layers, way more wiring, thousands of extra solder joints that can fail, and much higher power consumption. From every angle, the $8 \times \text{RAM1K}$ approach is clearly the cheaper and more sensible choice

Note that we assume the RAM8 and RAM1K are pre-built and manufactured in their respective optimalities, hence we don't discuss about the internals of the chip itself

Part 2: Hardware Multipliers and Custom Toolchains

1. Building the multiplier

The standard Hack ALU uses six control bits (**zx**, **nx**, **zy**, **ny**, **f**, **no**) and maps to 18 different operations, but there is no multiplier, hence, we design a **Wallace Tree Multiplier** that computes products and embed the .hdl it into the ALU(original) code.

Wallace Tree Multiplier: Idea

We use a tree of carry-save adders(CSAs) that operate with 3:2 I/O, to collapse all the rows of partial products, efficiently, until two rows remain. Finally we use an adder like a carry-look-ahead(CLA) adder to produce the final result

For two 16-bit inputs **a**[16] and **b**[16], the multiplication produces 16 partial products, each being **a** AND-ed with a single bit of **b** and shifted appropriately:

$$P_i = a \cdot b[i] \cdot 2^i, \quad i = 0, 1, \dots, 15$$

Reduction of addition operations

Stage	Rows	Action
0	16	16 partial products
1	16 → 11	Perform CSA on 5 sets of 3 + 1 leftover
2	11 → 8	Reduce again with CSAs
3	8 → 6	Continue reduction
4	6 → 4	Continue reduction
5	4 → 3	Continue reduction
6	3 → 2	Final CSA stage
Final	2 → 1	CLA produces final product

reduction simulation for 16 bits

NOTE: naive approach would chain 15 sequential additions ($O(n)$ adder delay). But the above method reduces this to $\lceil \log_{3/2}(16) \rceil = 7$ CSA stages plus one final CLA, providing a significant reduction in critical path delay.

Approach	Stages	Critical Path Delay
Naive (sequential adds)	$n - 1$ full additions in series	$O(n^2)$ ie., n stages $\times$ $O(n)$ carry propagation each
True Wallace Tree (CSA)	$\lceil \log_{3/2}(n) \rceil$ CSA stages + 1 CPA	$O(n)$ ie., CSA stages are $O(1)$ (no carry propagation), only the final CPA is $O(n)$

(n = bit width)

Adder Choice per Stage

- **Reduction stages (1–6):** We use **carry-save adders (CSAs)** This is because CSAs do not propagate carries, hence the reduction is done in $O(1)$ gate delay, regardless of the bit width.
- **Final stage:** The two remaining rows (sum and carry vectors) must be added with a **carry-look-ahead adder (CLA)**. A ripple-carry adder is also a simple choice, though a CLA has lesser delay at the cost of more gates. I have used CLA in my implementation

The Wallace tree reduces the number of sequential stages from 15 to 7+1, and since CSA stages are carry-free (constant delay), the overall latency is dominated by the single final CPA.

Multiplier HDL Implementation

The architecture is hence built as follows: $a[16], b[16] \rightarrow \text{Generate Partial Products} \rightarrow \text{CSA Reduction} \rightarrow \text{final CLA} \rightarrow \text{out}[16]$. There are two issues with implementing this straightforwardly:

- We have to **AND** a single bit of $b[16]$, with $a[16]$ bitwise, 16 times
- We have to each successive partial product by **one bit** incrementally, before feeding into the CSAs

These are not problems, but rather highly repeating units, if we had to write all the code in a single chip. I have workarounded this by writing the following 2 Helper Chips

8.0.1 Helper Chips

1. `And16Bit1.hdl` — Broadcasts a 1-bit signal across a 16-bit bus via AND.
2. `ShiftLeft1.hdl` — Shifts a 16-bit vector left by 1 position by pure wiring and no extra gates.

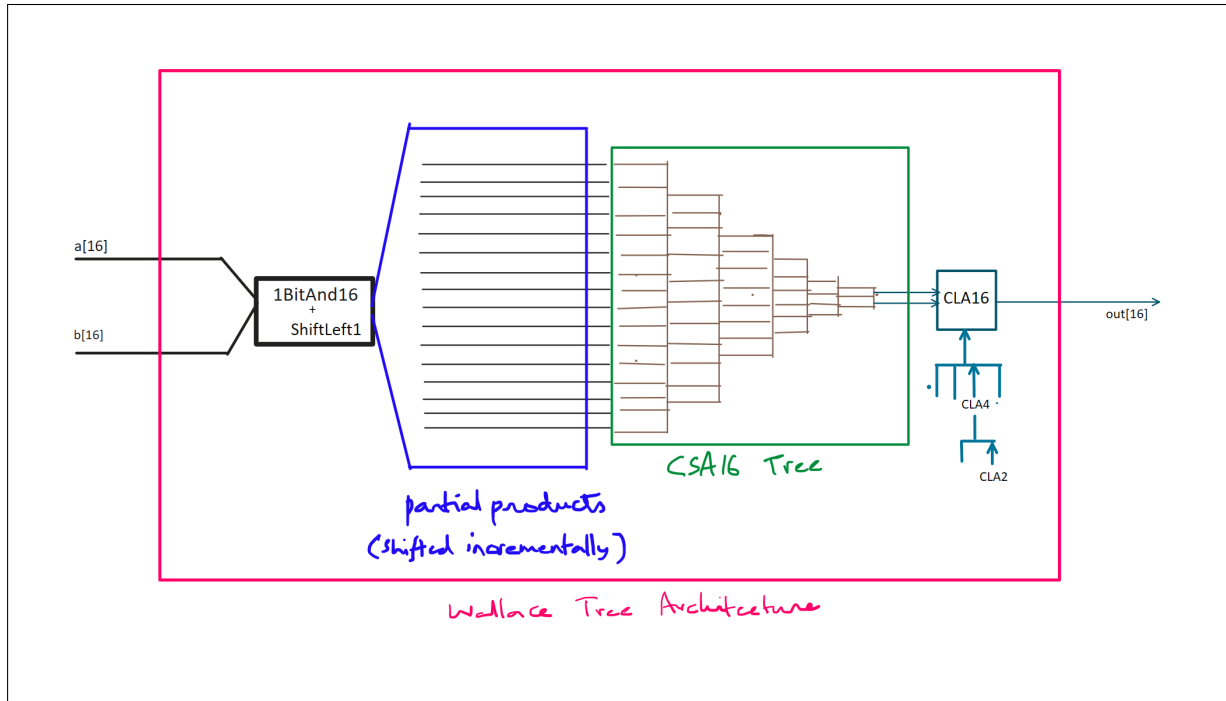
8.0.2 Adder Chips

1. `CSA16.hdl` — 16-bit Carry-Save Adder Which is a simple array of 16 FullAdders.
2. for the final `CLA16`, we build it recursively since writing it directly, like the Multiplier, has lots of units (and hence, lots of manual code). The hierarchy is built as follows
 - (a) `CLA2.hdl` — 2-bit CLA which is the base unit
 - (b) `CLA4.hdl` — 4-bit CLA = $2 \times \text{CLA2}$
 - (c) `CLA16.hdl` — 16-bit CLA = $4 \times \text{CLA4}$

8.0.3 Final Assembly

Used `And16Bit1.hdl` 16 times to generate partial products. Each partial product is incrementally *pre-shifted*, so all 16 partial products emerge already at their correct (shifted) bit positions and no post processing is needed.

The complete multiplier (`Multiplier.hdl`) composes these helpers into three stages: 16 partial products via `And16Bit1`, 6 stages of Wallace tree reduction via `CSA16 + ShiftLeft1` (14 of each), and one final addition via `CLA16`. Overflow beyond 16 bits is not considered. This makes the lower 16 bits correct for both positive and negative (via 2s complement)



16 Bit Wallace Tree Multiplier Architecture

2. Integrating into the ALU

given that the standard Hack ALU uses 6 control bits to encode 18 operations, we have $2^6 - 18 = 14$ unused sequences. we can use any of the unused mapping to support our MUL. For that, we can refer the used combinations of (`zx`, `nx`, `zy`, `ny`, `f`, `no`) by the standard ALU chip (see next page):

Existing ALU Operations (referred from textbook)

zx	nx	zy	ny	f	no	Output
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	$!x$
1	1	0	0	0	1	$!y$
0	0	1	1	1	1	$-x$
1	1	0	0	1	1	$-y$
0	1	1	1	1	1	$x + 1$
1	1	0	1	1	1	$y + 1$
0	0	1	1	1	0	$x - 1$
1	1	0	0	1	0	$y - 1$
0	0	0	0	1	0	$x + y$
0	1	0	0	1	1	$x - y$
0	0	0	1	1	1	$y - x$
0	0	0	0	0	0	$x \& y$
0	1	0	1	0	1	$x y$

Hence we can use 100000, 000001 etc to map the multiply instruction $x*y$.

Chosen Encoding for $x*y$

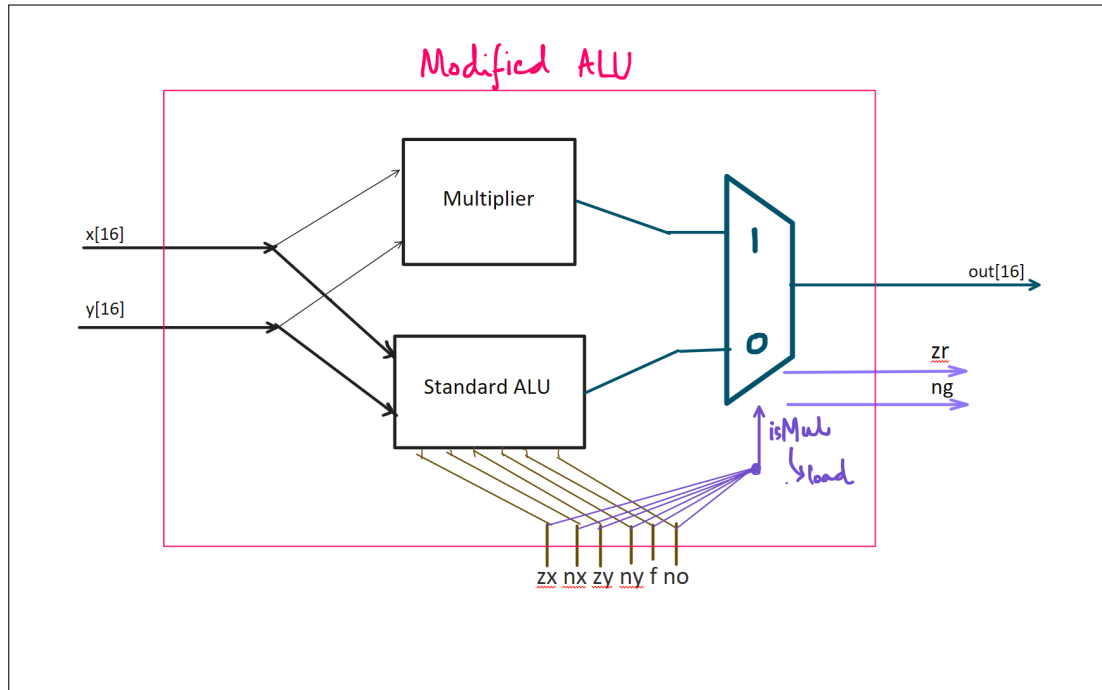
MUL Instruction Encoding

zx	nx	zy	ny	f	no
0	0	0	0	0	1

Rationale: From our reference table above, we can see that the combination 000001 is unused and hence we can use it to refer our multiplier

Modified ALU Architecture

The modification is simple. We can use a multiplexer (That fires when the MUL encoding, 000001 is set) to select between the standard ALU output and the multiplier output.



3. Testing the multiplier

- via ChatGPT, got an Exhaustive ALUregression.tst for the modified ALU, to verify with ALUregression.cmp to confirm the working of the multiplier and

Special Case discussion: Multiplicand of the Form 2^N or $2^N + 1$

We can simplify the multiplier when one of the multiplicand is 2^N or $2^N + 1$

Case 1: Multiplicand = 2^N

Multiplication by 2^N is equivalent to a **left shift by N positions**:

$$a \times 2^N = a \ll N$$

This does not require any adders as the result is simply the input a with N zeros appended on the right.

Case 2: Multiplicand = $2^N + 1$

we can expand the multiplication by $2^N + 1$:

$$a \times (2^N + 1) = a \times 2^N + a = (a \ll N) + a$$

ie., only a **single addition** of the shifted value and the original value.

Architectural Changes

if we can make the hardware know that one of the multiplicands is 2^N or $2^N + 1$ with some detector, we can design the multiplier to bypass the Wallace Tree with a simple bits shifter, falling back to the full multiplier otherwise.

9 Extra Credit

What is your recent favorite song? Finding Her

× × × End of Assignment × × ×